

PHẦN MỀM QUẢN LÝ HỢP ĐỒNG VÀ QUAN TRẮC MÔI TRƯỜNG
(*ENVIRONMENTAL ORDER MANAGEMENT SYSTEM*)
TÀI LIỆU TESTING

Testing Documentation for Environmental Order Management System

Phiên bản 1.0

Ngày tạo: 22/03/2026

MỤC LỤC

CHƯƠNG 1 – STATIC TESTING	3
<i>1.1 Tìm hiểu về Static Testing</i>	<i>3</i>
1.1.1 Static Testing là gì?	3
1.1.2 Tại sao Static Testing là cần thiết?	4
1.1.3 Các kỹ thuật Static Testing	4
1.1.4 Quy trình thực hiện Static Testing	6
1.1.5 Những lỗi chính được tìm thấy trong Static Testing	7
<i>1.2 Phân tích Static Testing cho Dự án ECOVA.....</i>	<i>8</i>
1.2.1 Module 1: Authentication & Authorization.....	7
1.2.2 Module 2: Employee Management.....	8
1.2.3 Module 3: Customer & Contract Management	8
1.2.4 Module 4: Planning	9
1.2.5 Module 5: Sampling & Testing	10
1.2.6 Module 6: Approval & Result Export	10
1.2.7 Module 7: Notification & Deadline.....	11
1.2.8 Module 8: Dashboard & AI	11
CHƯƠNG 2 – DYNAMIC TESTING – UNIT TESTING	15
<i>2.1 Tìm hiểu Unit Testing.....</i>	<i>17</i>
2.1.1 Unit Testing là gì?	17
2.1.2 Unit Testing có cần thiết không?.....	17
2.1.3 Unit Testing có nhược điểm gì?	17
<i>2.2 Implement Unit Testing cho ECOVA</i>	<i>17</i>
2.2.1 Module 1 - Authentication & Authorization: LoginUser().....	18
2.2.2 Module 2 - Employee Management: CreateEmployee()	18
2.2.3 Module 3 - Customer & Contract Management: CreateCustomer()	18
2.2.4 Module 4 - Planning: ValidateTestParameters()	18
2.2.5 Module 5 - Sampling & Testing: CheckWarningThreshold().....	18
2.2.6 Module 6 - Approval & Result Export: ExportPDF()	18

2.2.7 Module 7 - Notification & Deadline: SendNotificationEmail()	18
2.2.8 Module 8 - Dashboard & AI: PredictRenewalRisk()	18
KẾT LUẬN	19

CHƯƠNG 1. STATIC TESTING

Kiểm thử tĩnh (Static Testing) là phương pháp kiểm thử phần mềm dựa trên việc phân tích các tài liệu, thiết kế hệ thống, hoặc mã nguồn mà không cần thực thi chương trình. Điểm khác biệt quan trọng với kiểm thử động (Dynamic Testing) là static testing không yêu cầu chạy code. Phương pháp này được áp dụng để phát hiện những sai sót, khiếm khuyết trong giai đoạn sớm của quá trình phát triển, từ đó giảm chi phí và thời gian sửa lỗi sau này.

1.1 Tìm hiểu về Static Testing

1.1.1 Static Testing là gì?

Kiểm thử tĩnh là quá trình xem xét và đánh giá các tài liệu, thiết kế, cũng như mã nguồn một cách kỹ lưỡng để tìm ra những lỗi tiềm ẩn mà không cần phải chạy chương trình. Khác với kiểm thử động yêu cầu thực thi code để kiểm tra hành vi, static testing tập trung vào việc phân tích tĩnh các yêu cầu kỹ thuật, cấu trúc thiết kế, mã nguồn, kế hoạch kiểm thử và các kịch bản kiểm thử.

Static testing có thể được chia thành hai loại chính: (1) Kiểm thử thủ công thông qua các hoạt động review, trong đó con người trực tiếp xem xét tài liệu; (2) Kiểm thử tự động sử dụng các công cụ chuyên dụng để phân tích code và tìm kiếm các vấn đề tiềm ẩn.

Các loại tài liệu có thể được kiểm thử tĩnh bao gồm:

- Yêu cầu kỹ thuật (Functional và Non-functional Requirements)
- Tài liệu thiết kế hệ thống (System Design Documents)
- Mã nguồn chương trình (Source Code)
- Kế hoạch kiểm thử (Test Plan)
- Kịch bản thử nghiệm (Test Scripts)

- Các trường hợp kiểm thử (Test Cases)
- Tài liệu API và giao diện người dùng

1.1.2 Tại sao Static Testing là cần thiết?

Static testing đóng một vai trò cực kỳ quan trọng trong vòng đời phát triển phần mềm vì nhiều lý do chiến lược và kinh tế:

- **Phát hiện lỗi sớm:** Cho phép phát hiện các khiếm khuyết ngay từ giai đoạn yêu cầu hoặc thiết kế, trước khi chúng trở nên phức tạp và khó sửa chữa
- **Tiết kiệm chi phí:** Các khiếm khuyết được phát hiện sớm rẻ hơn rất nhiều so với những lỗi tìm thấy ở giai đoạn sau, đặc biệt là sau khi phần mềm được triển khai vào sản xuất
- **Cải thiện chất lượng thiết kế:** Thông qua quá trình review, thiết kế được cải tiến dần, mã trở nên dễ bảo trì hơn và dễ hiểu hơn
- **Tăng hiệu suất phát triển:** Giảm thời gian gỡ lỗi và tái lập, từ đó tăng tốc độ phát triển tính năng
- **Giảm chi phí kiểm thử:** Vì phát hiện lỗi sớm, giai đoạn kiểm thử động sẽ ít lỗi hơn để xử lý
- **Xác định các lỗi ẩn:** Một số khiếm khuyết khó phát hiện thông qua kiểm thử động có thể được phát hiện dễ dàng hơn thông qua static testing
- **Cải thiện truyền thông team:** Quá trình review giúp các thành viên team hiểu rõ hơn về yêu cầu, thiết kế và quy trình phát triển

1.1.3 Các kỹ thuật Static Testing

Có nhiều kỹ thuật khác nhau để thực hiện static testing, tùy thuộc vào loại tài liệu cần kiểm thử, mục tiêu của công ty, và các tài nguyên sẵn có:



Inspection (Kiểm tra chính thức): Là phương pháp kiểm thử chính thức nhất trong static testing. Một hoặc nhiều người kiểm duyệt sẽ xem xét tài liệu công việc một cách có hệ thống sử dụng danh sách kiểm tra được chuẩn bị sẵn. Mục tiêu là xác định mức độ hoàn thành của tài liệu và kiểm tra sự tuân thủ các tiêu chuẩn được đặt ra. Quá trình này thường yêu cầu chuẩn bị kỹ lưỡng và có các định mức rõ ràng.

Walk-through (Thuyết trình và thảo luận): Là kỹ thuật kiểm thử tương đối không chính thức hơn. Tác giả của tài liệu sẽ dẫn dắt các nhà xem xét thông qua tài liệu hoặc mã nguồn, giải thích logic và thiết kế. Những người tham gia có thể đặt ra các câu hỏi, nêu ra ý kiến, và ghi chú lại những điểm cần cải thiện. Kỹ thuật này giúp tăng cường hiểu biết chung và phát hiện những vấn đề logic.

Peer Reviews (Xem xét ngang cấp): Trong kỹ thuật này, các thành viên ngang cấp (developers, testers) sẽ xem xét công việc của nhau. Mục tiêu chính là đảm bảo code được viết tuân theo các tiêu chuẩn kỹ thuật, quy ước đặt ra, và các best practices. Xem xét ngang cấp thường tập trung vào các khía cạnh như hiệu suất, bảo mật, và khả năng bảo trì. Đây là một phương pháp hiệu quả để chia sẻ kiến thức và cải thiện chất lượng code.

Informal Reviews (Xem xét không chính thức): Là hình thức xem xét tài liệu một cách lỏng lẻo, không có quy trình chính thức hoặc danh sách kiểm tra. Các ý

kiến được đưa ra một cách tự do và không ràng buộc bởi bất kỳ tiêu chuẩn nào. Mặc dù không chính thức, nhưng phương pháp này vẫn có giá trị vì nó cho phép phân tích từ nhiều góc độ khác nhau.

1.1.4 Quy trình thực hiện Static Testing

Một quá trình static testing hiệu quả thường bao gồm 5 giai đoạn chính, mỗi giai đoạn có các hoạt động cụ thể và đầu ra rõ ràng:

Planning (Lập kế hoạch)

- **Xác định phạm vi review:** Quyết định những tài liệu hoặc phần nào cần được kiểm thử, mục tiêu cụ thể của review là gì.
- **Xác định các tiêu chí chất lượng:** Quyết định những đặc điểm nào sẽ được đánh giá (độ đầy đủ, tính nhất quán, tuân thủ tiêu chuẩn).
- **Ước lượng công sức:** Dự tính khoảng thời gian và tài nguyên cần thiết.
- **Lựa chọn kỹ thuật phù hợp:** Quyết định sử dụng inspection, walk-through, hay peer review.
- **Xác định vai trò:** Phân công người duyệt, người ghi chú, người dẫn dắt...

Initiate Review (Khởi động review)

- **Phân phối tài liệu:** Gửi sản phẩm cần review và các vật liệu hỗ trợ (danh sách kiểm tra, hướng dẫn, chuẩn mực) cho tất cả người tham gia.
- **Giải thích chi tiết:** Trình bày scope, mục tiêu, quy trình, vai trò của mỗi người, và những tiêu chí đánh giá.
- **Giải quyết câu hỏi:** Sẵn sàng trả lời những thắc mắc từ các thành viên tham gia.

Individual Review (Xem xét cá nhân)

- **Kiểm tra độc lập:** Mỗi người xem xét toàn bộ hoặc một phần của tài liệu một cách độc lập.
- **Ghi chú lỗi:** Ghi lại những lỗi tiềm ẩn, khuyến nghị cải thiện, và những câu hỏi cần làm rõ.

Issue Communication and Analysis (Thảo luận và phân tích)

- **Tập hợp phản hồi:** Thu thập tất cả các phát hiện từ các người xem xét.

- **Phân tích lỗi:** Xác định tính chất của mỗi lỗi, mức độ nghiêm trọng, và người chịu trách nhiệm sửa lỗi.
- **Đánh giá chất lượng:** Dựa trên các phát hiện, đánh giá toàn bộ chất lượng của tài liệu hoặc code.
- **Ra quyết định:** Quyết định liệu tài liệu có đạt tiêu chí chất lượng hay cần phải sửa chữa thêm.

Fixing and Report (Sửa chữa và báo cáo)

- **Tạo báo cáo lỗi:** Lập tài liệu chi tiết về tất cả các lỗi tìm thấy, mức độ và nguyên nhân.
- **Sửa lỗi:** Tiến hành sửa chữa những lỗi đã được xác định.
- **Thông báo tiến độ:** Cập nhật trạng thái sửa chữa cho tất cả những người liên quan.
- **Theo dõi và xác nhận:** Xác minh rằng những lỗi đã được sửa chữa đúng cách và tài liệu bây giờ đáp ứng các tiêu chí chất lượng.

1.1.5 Những lỗi chính được tìm thấy trong Static Testing

Thông qua quá trình static testing, các loại lỗi sau đây thường được phát hiện:

- **Không tuân thủ tiêu chuẩn:** Tài liệu hoặc code không theo đúng các tiêu chuẩn, quy ước, hoặc best practices đã được đặt ra.
- **Khó bảo trì:** Code viết phức tạp, khó đọc, hoặc thiếu comment khiến việc bảo trì trở nên khó khăn.
- **Lỗi trong thiết kế:** Logic thiết kế không hợp lý, cấu trúc dữ liệu không phù hợp, hoặc không tuân theo nguyên tắc thiết kế tốt.
- **Lỗi trong yêu cầu:** Yêu cầu mơ hồ, không rõ ràng, hay thậm chí là mâu thuẫn với nhau.
- **Thiếu tính nhất quán:** Có sự không nhất quán giữa các phần khác nhau của tài liệu hoặc hệ thống.
- **Lỗi bảo mật:** Phát hiện những lỗ hổng bảo mật tiềm ẩn trong thiết kế hoặc code.

- **Biên không được khai báo:** Sử dụng biên mà không khai báo hoặc khởi tạo.
- **Vi phạm ranh giới:** Không xử lý đúng các giá trị biên (edge cases).
- **Lỗi cú pháp:** Code không tuân theo cú pháp chính xác của ngôn ngữ lập trình.
- **Giao diện không nhất quán:** Giao diện người dùng không đồng nhất, gây nhầm lẫn cho người dùng.

1.2 Phân tích Static Testing cho Dự án ECOVA

1.2.1 Module 1: Authentication & Authorization

Chức năng: Đăng nhập, Quên mật khẩu (email), phân quyền (admin / phòng ban), FaceID login.

Các lỗi có thể xảy ra	Phân tích lỗi, trạng thái sau khi xảy ra lỗi	Hướng giải quyết
Username trống	Không thể đăng nhập, hệ thống báo lỗi ArgumentException: "Username and Password are required"	Kiểm tra đầu vào bằng <code>string.IsNullOrEmpty</code> , ràng buộc trường dữ liệu ở LoginForm khác null
Password trống	Không thể đăng nhập, hệ thống báo lỗi ArgumentException: "Username and Password are required"	Kiểm tra đầu vào bằng <code>string.IsNullOrEmpty</code> , ràng buộc trường dữ liệu ở LoginForm khác null
Username và password đều trống	Không thể đăng nhập, hệ thống báo lỗi ArgumentException	Ràng buộc trường dữ liệu ở UI khác null trước khi truyền xuống hàm Login

Đầu vào biến RoleCode không xác định	Đăng nhập được nhưng không phân quyền được	Định nghĩa lại các biến đầu vào, kiểm tra biến RoleCode đã định nghĩa tại AuthService
--------------------------------------	--	---

1.2.2 Module 2: Employee Management (Admin)

Chức năng: thêm nhân viên, sửa nhân viên, xóa nhân viên, tìm kiếm nhân viên.

Các lỗi có thể xảy ra	Phân tích lỗi, trạng thái sau khi xảy ra lỗi	Hướng giải quyết
Tên nhân viên (FullName) trống	Không thêm được nhân viên, báo lỗi ArgumentException: "FullName is required"	Kiểm tra đầu vào bằng string.IsNullOrEmpty, thêm ràng buộc Tên khác null
Email trống hoặc không chứa ký tự "@"	Không tạo được tài khoản, báo lỗi ArgumentException: "Valid Email is required"	Kiểm tra đầu vào, thêm ràng buộc Email khác null và bắt buộc chứa "@"
Số điện thoại trống	Không thêm được nhân viên	Kiểm tra đầu vào, thêm ràng buộc Số điện thoại khác null
Email hoặc Số điện thoại bị trùng lặp	Hệ thống từ chối lưu dữ liệu xuống Database	Dùng repository kiểm tra trước khi thêm và đặt ràng buộc UNIQUE trong SQL

1.2.3 Module 3: Customer & Contract (Phòng kinh doanh)

Chức năng: quản lý khách hàng, tạo hợp đồng, danh sách hợp đồng.

Các lỗi có thể xảy ra	Phân tích lỗi, trạng thái sau khi xảy ra lỗi	Hướng giải quyết
Thiếu mã khách hàng (CustomerID trống)	Không tạo được hợp đồng, hệ thống báo lỗi ArgumentException: "CustomerID is required"	Kiểm tra đầu vào bằng string.IsNullOrEmpty(contract.CustomerID) tại ContractService, thêm ràng buộc khác null
Ngày ký (SignedDate) trống hoặc không hợp lệ	Không tạo được hợp đồng, hệ thống báo lỗi ArgumentException: "Signed Date is required"	Kiểm tra đầu vào nếu contract.SignedDate == default(DateTime), thêm ràng buộc bắt buộc nhập
Ngày kết thúc (ValidTo) trống hoặc không hợp lệ	Không tạo được hợp đồng, hệ thống báo lỗi ArgumentException: "ValidTo Date is required"	Kiểm tra đầu vào nếu contract.ValidTo == default(DateTime), thêm ràng buộc bắt buộc nhập
Ngày kết thúc nhỏ hơn Ngày ký	Lưu sai logic hợp đồng, hệ thống báo lỗi ArgumentException	Kiểm tra điều kiện contract.ValidTo < contract.SignedDate trước khi thực thi việc lưu vào Database

1.2.4 Module 4: Planning (Phòng kế hoạch)

Chức năng: tạo nền mẫu, tạo vị trí mẫu, gán thông số đo.

Các lỗi có thể xảy ra	Phân tích lỗi, trạng thái sau khi xảy ra lỗi	Hướng giải quyết
-----------------------	--	------------------

Thiếu Nền mẫu (EnvironmentID trống)	Không tạo được vị trí mẫu, hệ thống báo lỗi ArgumentException: "Environment is required"	Kiểm tra đầu vào bằng string.IsNullOrEmpty(location.EnvironmentID) tại hàm CreateSampleLocation
Tên vị trí (LocationName) trống	Không tạo được vị trí, hệ thống báo lỗi ArgumentException: "Location Name is required"	Kiểm tra đầu vào bằng string.IsNullOrEmpty(location.LocationName), thêm ràng buộc Tên vị trí khác null
Danh sách thông số đo (parameters) rỗng hoặc null	Không gán được thông số đo, báo lỗi ArgumentException: "At least one parameter must be assigned"	Kiểm tra điều kiện parameters == null parameters.Count == 0 tại hàm AssignParameters

1.2.5 Module 5: Sampling & Testing (HT + PTN)

Chức năng: danh sách việc cần đo, nhập kết quả, cảnh báo vượt QCVN, sửa kết quả, in QR code.

Các lỗi có thể xảy ra	Phân tích lỗi, trạng thái sau khi xảy ra lỗi	Hướng giải quyết
Mã kết quả đo trống	Không lưu được kết quả	Kiểm tra đầu vào tại EnterResultForm, thêm ràng buộc Mã kết quả khác null

Giá trị đo trống	Không lưu được kết quả	Kiểm tra đầu vào, thêm ràng buộc Giá trị đo khác null
Đầu vào Giá trị đo không phải số	Không lưu được kết quả, hệ thống báo lỗi	Định nghĩa lại các biến đầu vào, ép kiểu dữ liệu về Float tại TestingService
Sửa kết quả nhưng thiếu Lý do	Lỗi lưu vết lịch sử (AuditLog)	Kiểm tra đầu vào tại EditResultReasonForm, thêm ràng buộc Lý do khác null

1.2.6 Module 6: Approval & Result Export

Chức năng: phòng kết quả kiểm tra, giám đốc phê duyệt, xuất PDF, gửi email.

Các lỗi có thể xảy ra	Phân tích lỗi, trạng thái sau khi xảy ra lỗi	Hướng giải quyết
Trạng thái kết quả chưa được Approve	Không xuất được file PDF	Kiểm tra biến trạng thái đầu vào, thêm điều kiện Status là Approved
Thiếu ID Hợp đồng/Kết quả	Không tìm thấy dữ liệu để xuất kết quả	Kiểm tra đầu vào, thêm ràng buộc ID khác null
Đầu vào từ chối duyệt thiếu Ghi chú	Không phản hồi được kết quả lại cho bộ phận khác	Kiểm tra đầu vào tại DirectorApprovalForm, thêm ràng buộc Ghi chú khác null
Đầu vào không xác định	Báo lỗi hệ thống	Định nghĩa lại các biến đầu vào, kiểm tra các biến đã định nghĩa tại ExportService

1.2.7 Module 7: Notification & Deadline

Chức năng: đơn hàng quá hạn, gửi email nhắc, nhắc tái ký hợp đồng.

Các lỗi có thể xảy ra	Phân tích lỗi, trạng thái sau khi xảy ra lỗi	Hướng giải quyết
Email người nhận (Recipient) trống	Không gửi được thông báo nhắc lịch	Kiểm tra đầu vào, thêm ràng buộc Email nhận khác null
Tiêu đề (Subject) trống	Email gửi đi bị lỗi định dạng	Kiểm tra đầu vào, thêm ràng buộc Tiêu đề khác null
Nội dung (Body) trống	Email gửi đi bị lỗi định dạng	Kiểm tra đầu vào, thêm ràng buộc Nội dung khác null
Số ngày quá hạn là biến không xác định	Không tính được lịch gửi email tự động	Định nghĩa lại các biến đầu vào, kiểm tra các biến đã định nghĩa tại NotificationService

1.2.8 Module 8: Dashboard & AI

Chức năng: biểu đồ đơn hàng, dự đoán tái ký, dự đoán ô nhiễm, thống kê & tìm kiếm.

Các lỗi có thể xảy ra	Phân tích lỗi, trạng thái sau khi xảy ra lỗi	Hướng giải quyết
Thời gian thống kê trống	Không hiển thị được biểu đồ đơn hàng	Kiểm tra đầu vào, thêm ràng buộc Từ ngày, Đến ngày khác null

Mã khách hàng trống	Không dự báo được tái ký hợp đồng	Kiểm tra đầu vào, thêm ràng buộc Mã khách hàng khác null
Số lượng hợp đồng/đơn hàng trống	Không có số liệu thống kê hiển thị	Kiểm tra đầu vào, thêm ràng buộc dữ liệu khác null
Đầu vào không xác định	Báo lỗi hệ thống, không hiện Dashboard	Định nghĩa lại các biến đầu vào, kiểm tra các biến đã định nghĩa

CHƯƠNG 2. DYNAMIC TESTING – UNIT TESTING

Dynamic Testing (Kiểm thử động) là phương pháp kiểm thử yêu cầu phải chạy (thực thi) mã nguồn để kiểm tra các hành vi và kết quả đầu ra của phần mềm. Trong phạm vi của dự án ECOVA, để đảm bảo chất lượng từ những thành phần nhỏ nhất, nhóm sẽ tập trung áp dụng mức độ đầu tiên và cốt lõi nhất của Dynamic Testing là Unit Testing (Kiểm thử đơn vị).

Kiểm thử đơn vị (Unit Testing) là phương pháp kiểm thử từng đơn vị phần mềm nhỏ (hàm, phương thức, lớp) một cách độc lập. Mục tiêu chính là xác định sự phù hợp của mỗi đơn vị với thông số kỹ thuật được thiết kế. Unit testing cho phép developer nhanh chóng phát hiện và sửa chữa lỗi, tăng tự tin trong việc refactor code, và đảm bảo mỗi phần của hệ thống hoạt động đúng theo yêu cầu.

2.1 Tìm hiểu Dynamic Testing

2.1.1 Dynamic Testing là gì?

Dynamic Testing (Kiểm thử động) là phương pháp kiểm thử yêu cầu phải chạy (thực thi) mã nguồn để kiểm tra các hành vi và kết quả đầu ra của phần mềm. Phương pháp này giúp xác nhận xem các đơn vị phần mềm riêng lẻ hoặc toàn bộ hệ thống có hoạt động đúng như thiết kế và yêu cầu hay không.

2.1.2 Sự khác nhau giữa Static Testing và Dynamic Testing là gì?

Sự khác biệt cơ bản giữa hai phương pháp này bao gồm:

(1) Mục đích: Static test nhằm mục đích ngăn ngừa lỗi, trong khi dynamic test tập trung vào việc điều trị lỗi;

(2) Thời điểm phát hiện: Static test giúp phát hiện khiếm khuyết sớm mà không cần thực thi chương trình, ngược lại dynamic test chỉ tìm thấy lỗi khi tiến hành thực thi code;

(3) Chi phí: Các khiếm khuyết được phát hiện bằng static test có chi phí sửa chữa thấp, trong khi lỗi tìm thấy qua dynamic test thường có chi phí sửa chữa cao hơn.

2.1.3 Các mức độ và phương pháp tiếp cận của Dynamic Testing?

Dựa trên mô hình vòng đời phát triển phần mềm, kiểm thử động được thực hiện qua các mức độ: Unit Test (Kiểm thử đơn vị), Integration Test (Kiểm thử tích hợp), System Test (Kiểm thử hệ thống) và Acceptance Test (Kiểm thử chấp nhận). Có hai phương pháp tiếp cận chính trong kiểm thử động:

(1) Black-box testing (Kiểm thử hộp đen): Tập trung vào đầu vào và đầu ra mà không can thiệp vào mã nguồn, bao gồm Functional testing (đảm bảo mỗi đơn vị phần mềm hoạt động chính xác theo đúng thiết kế) và Business testing (đảm bảo chương trình phần mềm hoạt động đúng theo các yêu cầu của người dùng);

(2) White-box testing (Kiểm thử hộp trắng): Phương pháp này dựa trên cấu trúc bên trong của mã nguồn và thường do chính các developer tự thực hiện. Nó bao gồm việc kiểm tra cú pháp bằng trình biên dịch để tránh lỗi, chạy code ở chế độ debug từng dòng qua các đường dẫn độc lập để đảm bảo mọi câu lệnh đều được thực thi ít nhất một lần, kiểm tra cấu trúc dữ liệu cục bộ nhằm duy trì tính toàn vẹn, kiểm tra các điều kiện biên và review tất cả các đường dẫn xử lý lỗi.

2.1.4 Các kỹ thuật Dynamic Testing?

Tùy thuộc vào phương pháp tiếp cận, kiểm thử động áp dụng các kỹ thuật cụ thể sau:

(1) Kỹ thuật Black box (Specification derived tests): Sử dụng Equivalence partitioning (phân lớp tương đương) để chia đầu vào của chương trình thành các lớp dữ liệu, giúp giảm số lượng test case cần thiết vì hành vi phần mềm được xem là tương đương đối với bất kỳ giá trị nào trong một phân vùng cụ thể. Ngoài ra còn có Boundary value analysis (phân tích giá trị biên), là trường hợp đặc biệt của phân vùng tương đương phù hợp với giá trị liên tục, tập trung kiểm tra tại các ranh giới giữa các phân vùng nơi dễ tồn tại lỗi nhất bằng các giá trị như min, min+, normal, max-, max;

(2) Kỹ thuật White box (Structural): Bao gồm Statement coverage (phủ cấp 1) đảm bảo mỗi câu lệnh chạy ít nhất 1 lần; Decision/branch coverage (phủ cấp 2) đảm bảo mỗi điểm quyết định đều được thực hiện ít nhất một lần ; và Path coverage đảm bảo mỗi đường chạy qua chương trình đều phải được đi qua ít nhất một lần bởi một test case nào đó.

2.2 Tìm hiểu Unit Testing

2.2.1 Unit Testing là gì?

Unit Test là phương pháp kiểm thử các đơn vị phần mềm riêng lẻ để xác định sự phù hợp của chúng với thông số kỹ thuật được thiết kế. Mỗi unit test bao gồm việc kiểm thử input (dữ liệu đầu vào), logic xử lý, và output (kết quả đầu ra) của một hàm hay phương thức cụ thể.

Các unit test thường được viết bởi developer cùng với code, hoặc ngay sau khi code được viết. Mục đích là chứng minh rằng từng hàm hoạt động đúng như mong đợi trước khi tích hợp với các phần khác.

2.1.2 Unit Testing có cần thiết không?

Unit testing rất cần thiết vì nhiều lý do:

(1) Phát hiện lỗi sớm: Lỗi được phát hiện ngay trong quá trình phát triển, dễ sửa chữa;

(2) Tăng tự tin: Khi code được kiểm thử kỹ lưỡng, developer tự tin hơn khi thực hiện các thay đổi;

(3) Cải thiện thiết kế: Viết unit test buộc developer suy nghĩ kỹ hơn về cách thiết kế hàm, dẫn đến code có cấu trúc tốt hơn, dễ bảo trì hơn;

(4) Giảm chi phí: Giảm chi phí sửa lỗi vì phát hiện sớm, giảm chi phí bảo trì vì code dễ hiểu hơn.

2.1.3 Unit Testing có nhược điểm gì?

Mặc dù lợi ích rõ ràng, unit testing vẫn có các nhược điểm:

(1) Tốn thời gian: Viết unit test mất thời gian bổ sung ngoài việc viết code;

(2) Yêu cầu kỹ năng: Không phải developer nào cũng biết cách viết unit test hiệu quả;

(3) Không đảm bảo 100%: Unit test chỉ kiểm thử từng đơn vị độc lập, không kiểm thử tương tác giữa các đơn vị hoặc hệ thống toàn bộ;

(4) Mock objects phức tạp: Khi hàm phụ thuộc vào các hàm khác, cần tạo mock objects giả lập, điều này có thể phức tạp.

2.3 Implement Unit Testing cho ECOVA

2.3.1 Module 1 - Authentication & Authorization: LoginUser()

[CODE IMPLEMENTATION]

2.3.2 Module 2 - Employee Management: CreateEmployee()

[CODE IMPLEMENTATION]

2.3.3 Module 3 - Customer & Contract Management: CreateCustomer()

[CODE IMPLEMENTATION]

2.3.4 Module 4 - Planning: ValidateTestParameters()

[CODE IMPLEMENTATION]

2.3.5 Module 5 - Sampling & Testing: CheckWarningThreshold()

[CODE IMPLEMENTATION]

2.3.6 Module 6 - Approval & Result Export: ExportPDF()

[CODE IMPLEMENTATION]

2.3.7 Module 7 - Notification & Deadline: SendNotificationEmail()

[CODE IMPLEMENTATION]

2.3.8 Module 8 - Dashboard & AI: PredictRenewalRisk()

[CODE IMPLEMENTATION]

KẾT LUẬN

Tài liệu testing này cung cấp hướng dẫn chi tiết và toàn diện về quy trình Static Testing và Unit Testing cho hệ thống Quản lý Hợp đồng và Quan trắc Môi trường ECOVA. Bằng cách thực hiện các bước testing một cách có hệ thống và kỹ lưỡng, nhóm phát triển có thể đảm bảo chất lượng code cao, giảm thiểu lỗi trong quá trình phát triển, và tăng độ tin cậy cũng như ổn định của ứng dụng khi triển khai vào sản xuất.

Các test case được cung cấp trong tài liệu này bao gồm 8 module. Mỗi module có các bảng phân tích lỗi chi tiết cho static testing và các sườn test case cho unit testing.